FH Salzburg
MultiMediaTechnology

Basics Game Development

GUI, GameDesign - Lab

Technology
Health
Media

Ui Button – Members and Constructor

```
internal class Button
{
    2 references
    public string TextString { get; private set; }
    7 references
    private Text Text { get;  set; }
    7 references
    public Color BackgroundColor { get; private set; }
    2 references
    public Color HoverColor { get; private set; }
    2 references
    public Vector2f Position { get; private set; }
    2 references
    public Vector2f Size { get; private set; }
    2 references
    public Color TextColor { get; private set; }
    3 references
    public uint FontSize { get; private set; }
    2 references
    public Font Font { get; private set; }
    3 references
    public Window renderWindow { get; private set; }
    10 references
    private RectangleShape ButtonRect { get; set; }

    public Action OnClick;
```

```
1 reference
public Button(string textString, Vector2f position, Vector2f size, Color backgroundColor, Color textColor,
    uint fontSize, Font font, Window window)
{
    TextString = textString;
    Position = position;
    Size = size;
    TextColor = textColor;
    FontSize = fontSize;
    Font = font;
    BackgroundColor = backgroundColor;
    renderWindow = window;

    HoverColor = new Color
    {
        R = (byte) (BackgroundColor.R * 0.9f),
        G = (byte) (BackgroundColor.G * 0.9f),
        B = (byte) (BackgroundColor.B * 0.9f),
        A = BackgroundColor.A
    };

    CreateBackground();
    CreateText();

    renderWindow.MouseButtonPressed += OnMouseButtonPressed;
    renderWindow.MouseMoved += OnMouseMoved;
}
```

Ui Button – Create Background and Text

```
1 reference
private void CreateBackground()
{
    ButtonRect = new RectangleShape(Size);
    ButtonRect.FillColor = BackgroundColor;
    ButtonRect.Position = Position;
}

1 reference
private void CreateText()
{
    Text = new Text(TextString, Font);
    Text.FillColor = TextColor;
    Text.CharacterSize = FontSize;
    Text.Origin = new Vector2f(x: Text.GetGlobalBounds().Width / 2f, y: FontSize / 2f);
    Text.Position = ButtonRect.Position +
                    new Vector2f(x: ButtonRect.GetGlobalBounds().Width / 2f, y: ButtonRect.GetGlobalBounds().Height / 2f);
}
```

Ui Button – Handle Mouse

```
1 reference
private void OnMouseMoved(object? sender, MouseMoveEventArgs e)
{
    ButtonRect.FillColor = ButtonRect.GetGlobalBounds().Contains(e.X, e.Y) ? HoverColor : BackgroundColor;
}

1 reference
private void OnMouseButtonPressed(object? sender, MouseButtonEventArgs e)
{
    if (e.Button == Mouse.Button.Left && ButtonRect.GetGlobalBounds().Contains(e.X, e.Y))
        OnClick?.Invoke();
}
```

```
1 reference
public void Draw(RenderWindow window)
{
    window.Draw(ButtonRect);
    window.Draw(Text);
}
```

Ui Button – Usage

```
button = new Button("Click Me!", new Vector2f(20, 20), new Vector2f(100, 40), Color.Blue, Color.Red, 20,
    AssetManager.Fonts["Arial"], window);

button.OnClick += () => { Console.WriteLine("Clicked"); };
```

Simple Generation from .txt

Think of simple data structures to generate content.

Read Data and place corresponding tiles

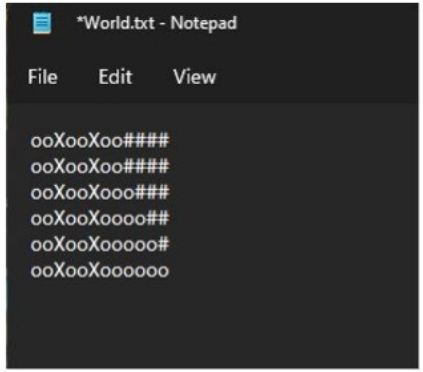
```
public void placeTiles(char[,] worldData, int tileSize)
{
    for (int i = 0; i < worldData.GetLength(1); i++)
    {
        for (int j = 0; j < worldData.GetLength(0); j++)
        {
            if (worldData[i, j] == 'X')
            {
                // Place Specific Tile on coordinate :  i * tileSize
                // zb  new Obstacle-GameObject( Vec2 pos )
            }
            else if (worldData[i, j] == 'o')
            {
                // ..
            }
            // usw..

        }
    }
```

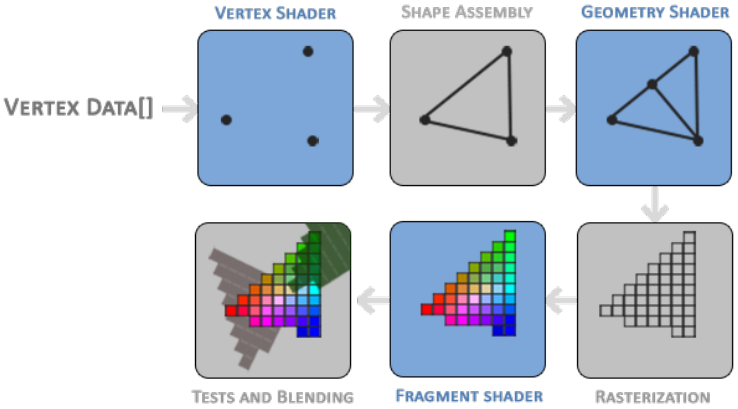
o =

=

X =



Shader



https://learnopengl.com/

Shader

- Load a Shader

```
Shader shader = new Shader("VertexShaderPath", "GeometryShaderPath" , "FragmentShaderPath" );
```

- Draw() using shader

```
RenderStates renderState = new RenderStates(shader);
```

```
window.Draw(drawable, renderState);
```

- Passing variables to the shader

```
renderState.Shader.SetUniform("PlayerScreenPos", playerScreenPos);
```

Shader – Simple view cone / top down flashlight

- Fragment Shader (File : LightShader.frag)

```
// pass value from script via uniforms
uniform vec2 PlayerScreenPos;
uniform float coneScale;

void main()
{
      // Calc Distance of Fragment
      // gl_TexCoord[0] -> tex coordiantes of texture with idx 0
      float dist = distance(gl_TexCoord[0] / coneScale , PlayerScreenPos / coneScale);

      // Pixel RGBA , A = distance
      vec4 pixel = vec4( 0, 0, 0, dist);

      // multiply it by the color
      gl_FragColor = gl_Color * pixel;
}
```

Shader – Simple view cone / top down flashlight

- Sample Shader loaded via AssetManager

```
// Load Shader

AssetManager.LoadShader("LightShader", null, null, @".\Assets\LightShader.frag");
AssetManager.LoadTexture("LightTexture",@".\Assets\LighTextureUV.png");
renderState= new RenderStates(AssetManager.Shaders["LightShader"]);
```

- Disclaimer :
  - A "empty" Texture (LightTextureUV.png) is used in order to get TexCoords / UVs to the Fragment Shader without using a Model-View-Projection Matrix and a Vertex Shader

Shader – Simple view cone / top down flashlight

- Sample use of the shader within SFML

```
public void DrawLight(uint ScreenWidth, uint ScreenHeight)
{
    var rectangle =
        new RectangleShape(new Vector2f(ScreenWidth, ScreenHeight));

    rectangle.Texture = AssetManager.Textures["LightTexture"];

    Vector2f playerScreenPos = new Vector2f(player.Position.X, player.Position.Y);

    playerScreenPos.X /= ScreenWidth;
    playerScreenPos.Y /= ScreenHeight;

    renderState.Shader.SetUniform("PlayerScreenPos", playerScreenPos);

    float coneScale = 0.5f;

    renderState.Shader.SetUniform("coneScale", coneScale);

    window.Draw(rectangle, renderState);
}
```

Simple "Physics" - Gravity

- Add a gravity

```
float gravity = 9.81f;
velocity.Y += gravity * deltaTime;
```

- Do movement / jump logic

```
if (InputManager.Instance.GetKeyPressed(Keyboard.Key.D))
{
    m_animationType = Animationtype.RunRight;
    velocity.X =  moveSpeed * deltaTime;
}
```

```
if(InputManager.Instance.GetKeyDown(Keyboard.Key.Space))
{
    velocity.Y = jumpForce;
}
```

- At the end of the frame add velocity to player position

```
Position += velocity * deltaTime;
```

TGUI

- TGUI.Net: Example code (C#)

Questions || Feedback || Wishes

FH Salzburg
MultiMediaTechnology

Technology
Health
Media

© 2025 Fachhochschule Salzburg GmbH
www.fh-salzburg.ac.at